Friedrich-Alexander-Universität
Erlangen-Nürnberg

Practical Skills in Computational Biology

Fundamentals of Bioimage Processing - Unit 1

Anna Möller[1, 2] **Dr. Anne Hartebrodt[1]** **Prof. Dr. Stefan Uderhardt[2]** **Prof. Dr. David B. Blumenthal[1]**

[1]Biomedical Network Science Lab, Department of Artificial Intelligence in Biomedical Engineering, Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU),

[2]Department of Medicine 3 – Rheumatology and Immunology, FAU and Universitätsklinikum Erlangen, 91054, Germany

09.06.2026

1. Introduction

2. NumPy Fundamentals

3. Image Input and Visualization

4. Indexing, Slicing, Cropping, and Masking

5. Image Arithmetics and Statistics

6. References

From Continuous to Digital Images

- Real-world image (continuous signal):

$$I(x, y) \in \mathbb{R}$$

- Digital image:
  - Discrete spatial sampling:

$$I[x, y], \quad x, y \in \mathbb{Z}$$

  - Quantized intensity values with bit depth b:

$$I[x, y] \in \{0, \ldots, 2^b - 1\}$$

- Two key steps:
  - Sampling (space)
  - Quantization (intensity)

Sampling: Continuous to Discrete Domain

- Continuous signal:

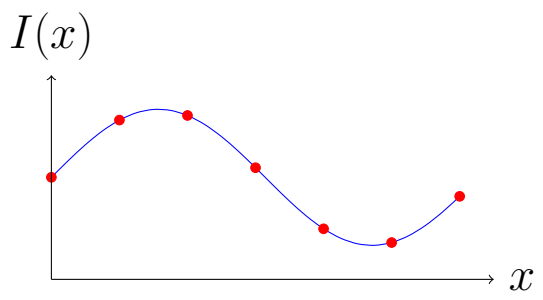
$$I(x) \in \mathbb{R}$$

- Sampling converts it to discrete points:

$$I[x], \quad x \in \mathbb{Z}$$

- Only values at specific positions are retained
- Controlled by the **sampling rate**

Sampling Example (1D Signal)



Blue: continuous signal Red: sampled points

Quantization: Continuous to Discrete Values

- Sampled signal still has continuous values:

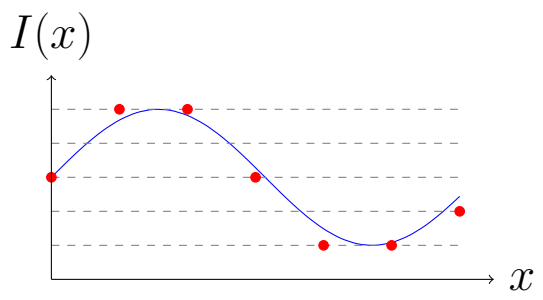
$$I[x] \in \mathbb{R}$$

- Quantization maps values to finite levels:

$$I[x] \in \{0, \ldots, 2^b - 1\}$$

- Depends on **bit depth** b
- Introduces **quantization error**

Quantization Example (1D Signal)



Blue: original signal Red: quantized values

1. Introduction

2. NumPy Fundamentals

3. Image Input and Visualization

4. Indexing, Slicing, Cropping, and Masking

5. Image Arithmetics and Statistics

6. References

Efficient Numerical Computing with NumPy

- NumPy provides the `ndarray` class for multidimensional arrays.
- Key advantages:
  - Efficient memory representation
  - Fast operations (implemented in C/Fortran)
  - Vectorized computations
- Standard import:
  - `import numpy as np`

Properties of NumPy Arrays

- **shape**: dimensions of the array
- **dtype**: data type (e.g., uint8, float32)
- **ndim**: number of dimensions

Example:

Image shape: (X, Y) or (X, Y, C) or (T, Z, C, X, Y)

1. Introduction

2. NumPy Fundamentals

3. Image Input and Visualization

4. Indexing, Slicing, Cropping, and Masking

5. Image Arithmetics and Statistics

6. References

Reading Images

- Images can be loaded using libraries such as `scikit-image`.
- Example function:
  - `skimage.io.imread()`
- Supports:
  - Local files
  - Relative paths
  - URLs

Displaying Images

- Common visualization tools:
  - `matplotlib.pyplot.imshow`
  - Specialized tools for microscopy images
- Important considerations:
  - Intensity scaling
  - Multi-channel handling
  - Colormaps (lookup tables)

1. Introduction

2. NumPy Fundamentals

3. Image Input and Visualization

4. Indexing, Slicing, Cropping, and Masking

5. Image Arithmetics and Statistics

6. References

Indexing

- Access individual pixels or elements:

$$I[x, y]$$

- Higher dimensions:

$$I[t, z, c, x, y]$$

- Properties:
  - Returns a single value or lower-dimensional array
  - Zero-based indexing (NumPy)

Slicing

- Extract subarrays using index ranges:

$$I[a : b,\ c : d]$$

- Interpretation:
  - Rows: a to b
  - Columns: c to d
- Properties:
  - Returns a **view** (no data copy)
  - Supports step sizes: $I[a : b : s, c : d : t]$

Cropping

- Cropping = selecting a region of interest (ROI):

$$I_{\text{crop}} = I[a : b,\ c : d]$$

- Purpose:
  - Focus on relevant image regions
  - Reduce data size
- Typical use:
  - Object extraction
  - Preprocessing step

Masking

- Define a boolean mask:

$$M[x, y] = (I[x, y] > T)$$

- Apply mask:

$$I_{\text{masked}} = I[M]$$

- Applications:
  - Thresholding
  - Region selection

1. Introduction

2. NumPy Fundamentals

3. Image Input and Visualization

4. Indexing, Slicing, Cropping, and Masking

5. Image Arithmetics and Statistics

6. References

Pixel-wise Arithmetic

- Images are numerical arrays:

$$I[x, y] \in \mathbb{R}$$

- Operations are applied element-wise:

$$I_{\text{new}}[x, y] = f(I[x, y])$$

- Examples:
  - Addition: $I + c$
  - Scaling: αI
  - Nonlinear: $\exp(I)$

Combining Images

- Combine images of same shape:

$$I_{\text{result}}[x, y] = I_1[x, y] + I_2[x, y]$$

- General form:

$$I_{\text{result}} = \alpha I_1 + \beta I_2$$

- Applications:
  - Image blending
  - Noise reduction (averaging)

Statistical Measures

- Number of pixels: N
- Mean intensity:

$$\mu = \frac{1}{N} \sum_{x,y} I[x, y]$$

- Variance:

$$\sigma^2 = \frac{1}{N} \sum_{x,y} (I[x, y] - \mu)^2$$

- Min / Max:

$$\min_{x,y} I[x, y], \quad \max_{x,y} I[x, y]$$

Histograms

- Distribution of intensity values
- Frequency of value k:

$$h[k] = \sum_{x,y} \mathbf{1}\{I[x,y] = k\}$$

- Normalized histogram:

$$p[k] = \frac{h[k]}{N}$$

- Interpretation:
  - Contrast
  - Dynamic range usage

1. Introduction

2. NumPy Fundamentals

3. Image Input and Visualization

4. Indexing, Slicing, Cropping, and Masking

5. Image Arithmetics and Statistics

6. References

References

[1] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. 3rd ed. Pearson, 2008. ISBN: 978-0131687288.

[2] Robert Haase et al. *Bio-image Analysis Notebooks*. Accessed: 2026-03-25. 2023. URL: https://haesleinhuepf.github.io/BioImageAnalysisNotebooks/.

[3] NumPy Developers. *NumPy Documentation*. 2026. URL: https://numpy.org.

[4] scikit-image development team. *scikit-image Documentation*. 2026. URL: https://scikit-image.org.

[5] Guillaume Witz. *Image Processing with Python for Beginners*. Accessed: 2026-03-25. 2021. URL: https://guiwitz.github.io/PyImageCourse_beginner/README.html.